UNIVERSIDAD CATÓLICA DE TEMUCO
INGENIERÍA CIVIL EN INFORMÁTICA
FACULTAD DE INGENIERÍA

UNIVERSIDAD CATÓLICA DE TEMUCO
Facultad de Ingeniería

Proyecto: Diseño e implementación de un analizador sintáctico LL(1)

CyberGuard DSL: lenguaje de reglas de ciberseguridad

Asignatura: Teoría de la Computación – INFO1148
Proyecto grupal – Análisis sintáctico y gramáticas libres de contexto

Integrantes

Ferrer Retamal, Christián
Escares Ortiz, Eduardo
Benavides Monsálvez, Patricio

Carrera:	Ingeniería Civil Informática
Docente:	Marcos Antonio Levano Huamaccto
Semestre:	2026-2
Fecha de entrega:	1 de noviembre de 2026

Temuco – Chile

Índice

1. Introducción y dominio seleccionado **2**

2. Especificación del lenguaje **2**
2.1. Propósito 2
2.2. Alfabeto relevante 2
2.3. Categorías de tokens 3
2.4. Ejemplos de sentencias 3

3. Gramática libre de contexto **4**

4. Análisis de la gramática **5**
4.1. Ambigüedad 5
4.2. Recursión por la izquierda 5
4.3. Factorización por la izquierda 5

5. Conjuntos FIRST y FOLLOW **5**

6. Tabla LL(1) **6**

7. Diseño e implementación del parser **8**
7.1. Relación entre la gramática y el código 9

8. Árbol sintáctico y AST **9**

9. Manejo de errores sintácticos **10**

10. Casos de prueba **11**
10.1. Extensiones creativas 11
10.2. Resultados 11

11. Conclusiones **13**

12. Anexos **14**
12.1. Repositorio 14
12.2. Estructura del repositorio 14
12.3. Demostración funcional 14

1. Introducción y dominio seleccionado

En la administración de infraestructuras de red y en los Centros de Operaciones de Seguridad (*SOC*), los analistas necesitan definir políticas declarativas de respuesta ante incidentes (fuerza bruta, escaneo de puertos, DDoS, exfiltración de datos) sin escribir scripts imperativos extensos.

El dominio elegido corresponde a **reglas simples de ciberseguridad y control de firewall**. Para ello se diseñó **CyberGuard DSL**, un lenguaje de dominio específico cuyas reglas evalúan condiciones lógicas y relacionales sobre métricas de telemetría (intentos de autenticación, puertos, volumen de tráfico, estado del firewall, listas blancas) y disparan una o más acciones defensivas, con o sin parámetros, tanto cuando la condición se cumple (`THEN`) como en su flujo alternativo (`ELSE`).

El objetivo del proyecto es aplicar gramáticas libres de contexto, conjuntos FIRST y FOLLOW, análisis LL(1), construcción de árboles sintácticos y detección de errores, mediante un parser implementado sin generadores automáticos. El código fuente completo está disponible en el repositorio del proyecto (ver Anexo 12).

2. Especificación del lenguaje

2.1. Propósito

Permitir escribir programas formados por una o más reglas de seguridad con la estructura:

```
WHEN <condicion> THEN <lista_acciones> [ELSE <lista_acciones>] ;
```

2.2. Alfabeto relevante

El alfabeto Σ está constituido por:

- letras ASCII `A-Z`, `a-z` y el guion bajo `_`;
- dígitos decimales `0-9`;
- símbolos relacionales `>`, `<`, `=`, `!`;
- puntuación y agrupación `;`, `,`, `(`, `)`;
- blancos (espacio, tabulación, salto de línea) y delimitadores de comentario `#` y `//`.

2.3. Categorías de tokens

Categoría	Token	Lexemas / expresión	Descripción
Palabras reservadas (8)	`WHEN, THEN, ELSE, AND, OR, NOT, TRUE, FALSE`	Insensibles a mayúsculas y minúsculas	Control condicional, operadores booleanos y literales lógicos.
Identificadores	`ID`	`[A-Za-z_][A-Za-z0-9_]*`	Variables de telemetría (`intentos_ssh`) y nombres de acciones (`bloquear_ip`).
Valores numéricos	`NUM`	`\d +`	Enteros no negativos (`22`, `443`).
Valores booleanos	`TRUE, FALSE`	`TRUE, FALSE`	Banderas de seguridad.
Operadores relacionales (6)	`OP_REL`	`>=, <=, ==, !=, >, <`	Comparadores numéricos y de igualdad.
Operadores lógicos (3)	`AND, OR, NOT`	`AND, OR, NOT`	Conjunción, disyunción y negación.
Separadores y agrupadores	`PUNTO_COMA, COMA, PARENT_IZQ, PARENT_DER`	`;, ,, (, )`	Fin de regla, separador de acciones, agrupación y parámetros.

Cuadro 1: Categorías de tokens de CyberGuard DSL.

2.4. Ejemplos de sentencias

Cinco sentencias válidas.

1. `WHEN intentos_login > 5 THEN bloquear ip_origen;`
2. `WHEN puerto_destino == 22 AND ip_verificada == FALSE THEN cerrar_puerto(22);`
3. `WHEN trafico_mb >= 1000 OR conexiones_activas >= 500 THEN limitar_ancho_banda(100), alertar soc_admin;`
4. `WHEN NOT firewall_activo == TRUE THEN activar firewall_perimetral, registrar_incidente;`
5. `WHEN (intentos_ssh > 3 OR intentos_rdp > 3) AND ip_interna == FALSE THEN aislar_host(ip_origen) ELSE registrar_acceso;`

Cinco sentencias inválidas.

1. `intentos_login > 5 THEN bloquear ip_origen;` *(falta `WHEN`).*
2. `WHEN intentos_login > THEN bloquear ip_origen;` *(falta el valor tras `>`).*

3. `WHEN intentos_login > 5 bloquear ip_origen;` *(falta* ***THEN****).*
4. `WHEN (intentos_login > 5 AND puerto == 22 THEN bloquear ip_origen;` *(paréntesis sin cerrar).*
5. `WHEN intentos_login > 5 THEN bloquear ip_origen` *(falta* ***;*** *final).*

3. Gramática libre de contexto

Sea $G = (V_N, V_T, P, S)$ la gramática en su versión final LL(1).

- **Símbolo inicial:** ⟨programa⟩.
- **No terminales (16):** ⟨programa⟩, ⟨resto_programa⟩, ⟨regla⟩, ⟨condicion⟩, ⟨resto_or⟩, ⟨termino_and⟩, ⟨resto_and⟩, ⟨factor_not⟩, ⟨primario⟩, ⟨comparacion⟩, ⟨valor⟩, ⟨lista_acciones⟩, ⟨resto_acciones⟩, ⟨accion⟩, ⟨complemento_accion⟩, ⟨clausula_else⟩.
- **Terminales (15):** WHEN, THEN, ELSE, AND, OR, NOT, TRUE, FALSE, ID, NUM, OP_REL, PARENT_IZQ, PARENT_DER, COMA, PUNTO_COMA.

Producciones

N.ž	Producción
(01)	⟨programa⟩ → ⟨regla⟩ ⟨resto_programa⟩
(02)	⟨resto_programa⟩ → ⟨regla⟩ ⟨resto_programa⟩ | ε
(03)	⟨regla⟩ → WHEN ⟨condicion⟩ THEN ⟨lista_acciones⟩ ⟨clausula_else⟩ PUNTO_COMA
(04)	⟨condicion⟩ → ⟨termino_and⟩ ⟨resto_or⟩
(05)	⟨resto_or⟩ → OR ⟨termino_and⟩ ⟨resto_or⟩ | ε
(06)	⟨termino_and⟩ → ⟨factor_not⟩ ⟨resto_and⟩
(07)	⟨resto_and⟩ → AND ⟨factor_not⟩ ⟨resto_and⟩ | ε
(08)	⟨factor_not⟩ → NOT ⟨factor_not⟩ | ⟨primario⟩
(09)	⟨primario⟩ → PARENT_IZQ ⟨condicion⟩ PARENT_DER | ⟨comparacion⟩
(10)	⟨comparacion⟩ → ID OP_REL ⟨valor⟩
(11)	⟨valor⟩ → NUM | TRUE | FALSE | ID
(12)	⟨lista_acciones⟩ → ⟨accion⟩ ⟨resto_acciones⟩
(13)	⟨resto_acciones⟩ → COMA ⟨accion⟩ ⟨resto_acciones⟩ | ε
(14)	⟨accion⟩ → ID ⟨complemento_accion⟩
(15)	⟨complemento_accion⟩ → ID | PARENT_IZQ ⟨valor⟩ PARENT_DER | ε
(16)	⟨clausula_else⟩ → ELSE ⟨lista_acciones⟩ | ε

Cuadro 2: Producciones de la gramática final.

4. Análisis de la gramática

4.1. Ambigüedad

Una formulación inicial ingenua de las condiciones sería:

$$\langle \texttt{condicion} \rangle \to \langle \texttt{condicion} \rangle \ \texttt{OR} \ \langle \texttt{condicion} \rangle \mid \langle \texttt{condicion} \rangle \ \texttt{AND} \ \langle \texttt{condicion} \rangle \mid \texttt{NOT} \ \langle \texttt{condicion} \rangle \mid \texttt{(} \ \langle \texttt{condicion} \rangle \ \texttt{)} \mid \langle \texttt{comparacion} \rangle$$

Esta gramática es **ambigua**: la sentencia `a > 1 OR b < 2 AND c == 3` admite dos árboles de derivación, `(a > 1 OR b < 2) AND c == 3` y `a > 1 OR (b < 2 AND c == 3)`.

Solución: se estratificó por niveles de precedencia, de menor a mayor: (1) ⟨`condicion`⟩ maneja `OR`; (2) ⟨`termino_and`⟩ maneja `AND`; (3) ⟨`factor_not`⟩ maneja `NOT` (recursiva por la derecha); (4) ⟨`primario`⟩ maneja paréntesis y comparaciones atómicas.

4.2. Recursión por la izquierda

La estratificación con asociatividad izquierda produciría recursión izquierda directa ($A \to A\alpha \mid \beta$):

- ⟨`condicion`⟩ → ⟨`condicion`⟩ `OR` ⟨`termino_and`⟩ | ⟨`termino_and`⟩
- ⟨`termino_and`⟩ → ⟨`termino_and`⟩ `AND` ⟨`factor_not`⟩ | ⟨`factor_not`⟩
- ⟨`lista_acciones`⟩ → ⟨`lista_acciones`⟩ `COMA` ⟨`accion`⟩ | ⟨`accion`⟩

Se aplicó la transformación estándar $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$, obteniendo las producciones (04)–(07) y (12)–(13).

4.3. Factorización por la izquierda

Existían dos puntos con prefijo común que habrían generado conflictos FIRST/FIRST:

1. ⟨**`programa`**⟩**:** ⟨`programa`⟩ → ⟨`regla`⟩ ⟨`programa`⟩ | ⟨`regla`⟩. Se factorizó en las producciones (01) y (02).
2. ⟨**`accion`**⟩**:** ⟨`accion`⟩ → `ID` `ID` | `ID` `PARENT_IZQ` ⟨`valor`⟩ `PARENT_DER` | `ID`. Se factorizó en las producciones (14) y (15).

5. Conjuntos FIRST y FOLLOW

Los conjuntos se calcularon mediante el algoritmo iterativo de punto fijo, implementado en `src/grammar_ll1.py`, sobre la gramática factorizada y sin recursión izquierda.

No terminal	FIRST	FOLLOW
⟨programa⟩	WHEN	$
⟨resto_programa⟩	WHEN, ε	$
⟨regla⟩	WHEN	WHEN, $
⟨condicion⟩	ID, NOT, PARENT_IZQ	PARENT_DER, THEN
⟨resto_or⟩	OR, ε	PARENT_DER, THEN
⟨termino_and⟩	ID, NOT, PARENT_IZQ	OR, PARENT_DER, THEN
⟨resto_and⟩	AND, ε	OR, PARENT_DER, THEN
⟨factor_not⟩	ID, NOT, PARENT_IZQ	AND, OR, PARENT_DER, THEN
⟨primario⟩	ID, PARENT_IZQ	AND, OR, PARENT_DER, THEN
⟨comparacion⟩	ID	AND, OR, PARENT_DER, THEN
⟨valor⟩	FALSE, ID, NUM, TRUE	AND, OR, PARENT_DER, THEN
⟨lista_acciones⟩	ID	ELSE, PUNTO_COMA
⟨resto_acciones⟩	COMA, ε	ELSE, PUNTO_COMA
⟨accion⟩	ID	COMA, ELSE, PUNTO_COMA
⟨complemento_accion⟩	ID, PARENT_IZQ, ε	COMA, ELSE, PUNTO_COMA
⟨clausula_else⟩	ELSE, ε	PUNTO_COMA

Cuadro 3: Conjuntos FIRST y FOLLOW.

Justificaciones principales.

- FOLLOW(⟨condicion⟩) = {THEN, PARENT_DER}, porque ⟨condicion⟩ aparece antes de THEN en (03) y antes de) en (09).
- FOLLOW(⟨valor⟩) hereda FOLLOW(⟨comparacion⟩) por (10) e incluye PARENT_DER por (15).
- FOLLOW(⟨accion⟩) = {COMA, ELSE, PUNTO_COMA}: viene de FIRST(⟨resto_acciones⟩) $\setminus \{\varepsilon\}$ y de FOLLOW(⟨lista_acciones⟩).
- FOLLOW(⟨clausula_else⟩) = {PUNTO_COMA}, ya que precede a ; en (03).

6. Tabla LL(1)

A partir de FIRST y FOLLOW se construye la tabla $M[A, a]$: para cada producción $A \to \alpha$, se asigna a $M[A, a]$ para todo $a \in \text{FIRST}(\alpha)$ y, si $\varepsilon \in \text{FIRST}(\alpha)$, para todo $a \in \text{FOLLOW}(A)$.

- **Celdas definidas:** 39.
- **Conflictos:** 0 (ninguna celda tiene más de una producción).
- **Conclusión:** la gramática **es LL(1)**, pues (i) para toda $A \to \alpha \mid \beta$ se cumple $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ y (ii) para todo A anulable, $\text{FIRST}(A) \cap \text{FOLLOW}(A) = \emptyset$.

Cuadro 4: Entradas activas de la tabla predictiva $M[A, a]$.

No terminal	**Terminal**	**Producción**
⟨programa⟩	WHEN	⟨regla⟩ ⟨resto_programa⟩
⟨resto_programa⟩	WHEN	⟨regla⟩ ⟨resto_programa⟩
⟨resto_programa⟩	$	ε
⟨regla⟩	WHEN	WHEN ⟨condicion⟩ THEN ⟨lista_acciones⟩ ⟨clausula_else⟩ PUNTO_COMA
⟨condicion⟩	ID, NOT, PARENT_IZQ	⟨termino_and⟩ ⟨resto_or⟩
⟨resto_or⟩	OR	OR ⟨termino_and⟩ ⟨resto_or⟩
⟨resto_or⟩	PARENT_DER, THEN	ε
⟨termino_and⟩	ID, NOT, PARENT_IZQ	⟨factor_not⟩ ⟨resto_and⟩
⟨resto_and⟩	AND	AND ⟨factor_not⟩ ⟨resto_and⟩
⟨resto_and⟩	OR, PARENT_DER, THEN	ε
⟨factor_not⟩	NOT	NOT ⟨factor_not⟩
⟨factor_not⟩	ID, PARENT_IZQ	⟨primario⟩
⟨primario⟩	PARENT_IZQ	PARENT_IZQ ⟨condicion⟩ PARENT_DER
⟨primario⟩	ID	⟨comparacion⟩
⟨comparacion⟩	ID	ID OP_REL ⟨valor⟩
⟨valor⟩	NUM	NUM
⟨valor⟩	TRUE	TRUE
⟨valor⟩	FALSE	FALSE
⟨valor⟩	ID	ID
⟨lista_acciones⟩	ID	⟨accion⟩ ⟨resto_acciones⟩
⟨resto_acciones⟩	COMA	COMA ⟨accion⟩ ⟨resto_acciones⟩
⟨resto_acciones⟩	ELSE, PUNTO_COMA	ε
⟨accion⟩	ID	ID ⟨complemento_accion⟩
⟨complemento_accion⟩	ID	ID
⟨complemento_accion⟩	PARENT_IZQ	PARENT_IZQ ⟨valor⟩ PARENT_DER
⟨complemento_accion⟩	COMA, ELSE, PUNTO_COMA	ε
⟨clausula_else⟩	ELSE	ELSE ⟨lista_acciones⟩
⟨clausula_else⟩	PUNTO_COMA	ε

7. Diseño e implementación del parser

El sistema se implementó en Python, sin generadores automáticos de parsers y sin dependencias externas (solo biblioteca estándar). Se organiza en los siguientes módulos:

1. `src/lexer.py`: analizador léxico con expresiones regulares compiladas. Reconoce palabras reservadas, identificadores, números, operadores relacionales, paréntesis, comas y comentarios (`#` y `//`), y guarda fila, columna y posición de cada token.

2. `src/grammar_ll1.py`: modela `NO_TERMINALES`, `TERMINALES` y `PRODUCCIONES`, e implementa `calcular_first()`, `calcular_follow()` y `construir_tabla_ll1()`.

3. `src/parser.py`: contiene dos implementaciones coherentes entre sí.

   - `ParserDescensoRecursivo`: un método por cada uno de los 16 no terminales (`parse_programa`, `parse_regla`, `parse_condicion`, etc.). Decide qué producción expandir mirando solo el token de preanálisis ($k = 1$) y construye el árbol sintáctico concreto y el AST.
   - `ParserTablaLL1`: analizador predictivo no recursivo con pila explícita y la tabla $M[A, a]$.

4. `src/ast_visualizer.py`: exporta árboles como texto jerárquico, Graphviz (`.dot`), Mermaid y SVG.

5. `tests/test_suite.py` y `main.py`: batería automatizada de 30 casos y CLI de demostración.

Cada método `parse_X` del descenso recursivo corresponde a las producciones de ⟨`X`⟩; los no terminales anulables (por ejemplo ⟨`resto_or`⟩) se resuelven por el conjunto FOLLOW, tal como indica la tabla de la sección 6.

7.1. Relación entre la gramática y el código

Producciones	Método del parser	Decisión con el token actual
(01)–(02) ⟨programa⟩, ⟨resto_programa⟩	parse_programa, parse_resto_programa	Si es WHEN, se analiza otra regla; si es $, se termina.
(03) ⟨regla⟩	parse_regla	Consume WHEN, la condición, THEN, las acciones, el ELSE opcional y ;.
(04)–(05) ⟨condicion⟩, ⟨resto_or⟩	parse_condicion, parse_resto_or	Mientras el token sea OR, continúa; con) o THEN aplica ε.
(06)–(07) ⟨termino_and⟩, ⟨resto_and⟩	parse_termino_and, parse_resto_and	Mientras el token sea AND, continúa; con OR,) o THEN aplica ε.
(08) ⟨factor_not⟩	parse_factor_not	NOT: recursión; ID o (: delega en ⟨primario⟩.
(09) ⟨primario⟩	parse_primario	(: agrupación; ID: comparación.
(10)–(11) ⟨comparacion⟩, ⟨valor⟩	parse_comparacion, parse_valor	Exige ID, OP_REL y luego NUM, TRUE, FALSE o ID.
(12)–(13) ⟨lista_acciones⟩, ⟨resto_acciones⟩	parse_lista_acciones, parse_resto_acciones	Mientras el token sea COMA, continúa; con ELSE o ; aplica ε.
(14)–(15) ⟨accion⟩, ⟨complemento_accion⟩	parse_accion, parse_complemento_accion	ID: objeto; (: parámetro; COMA, ELSE o ;: ε.
(16) ⟨clausula_else⟩	parse_clausula_else	ELSE: lista de acciones; ;: ε.

Cuadro 5: Correspondencia entre producciones y métodos de ParserDescensoRecursivo.

Además, ParserTablaLL1 usa directamente la tabla $M[A, a]$ calculada por construir_tabla_ll1(). Se comprobó que sus resultados de aceptación y rechazo coinciden con los esperados en los 30 casos de prueba.

Ejecución

```
python main.py                      # demostracion completa
python main.py -e "WHEN a > 1 THEN bloquear ip;"
python tests/test_suite.py          # 30 casos de prueba
python src/grammar_ll1.py           # FIRST, FOLLOW y conflictos
```

8. Árbol sintáctico y AST

Para la sentencia válida representativa:

```
WHEN (intentos_ssh > 3 OR puerto_destino == 22) AND NOT ip_confiable == TRUE
THEN bloquear_ip(ip_origen), alertar soc_admin ELSE registrar_acceso;
```

el AST generado es:

```
Programa
‘-- Regla
    |-- Condicion
    |   ‘-- OpLogico: AND
    |       |-- OpLogico: OR
    |       |   |-- Comparacion: >
    |       |   |   |-- Identificador: intentos_ssh
    |       |   |   ‘-- Numero: 3
    |       |   ‘-- Comparacion: ==
    |       |       |-- Identificador: puerto_destino
    |       |       ‘-- Numero: 22
    |       ‘-- OpUnario: NOT
    |           ‘-- Comparacion: ==
    |               |-- Identificador: ip_confiable
    |               ‘-- Booleano: TRUE
    |-- AccionesTHEN
    |   |-- Accion: bloquear_ip
    |   |   ‘-- Parametro
    |   |       ‘-- Identificador: ip_origen
    |   ‘-- Accion: alertar
    |       ‘-- Objetivo: soc_admin
    ‘-- AccionesELSE
        ‘-- Accion: registrar_acceso
```

La ejecución de `python main.py` genera además las versiones gráficas del AST y del árbol sintáctico concreto en la carpeta `outputs/` (`ast_ejemplo.svg`, `arbol_sintactico_ejemplo.svg` y sus archivos `.dot`).

9. Manejo de errores sintácticos

El analizador entrega mensajes que indican **qué se esperaba**, **qué se encontró** y la **posición** (número de token, línea y columna). Ejemplos obtenidos con el programa:

- *Falta de valor tras un operador relacional:*
  `Error sintáctico: se esperaba un valor (NUM, TRUE, FALSE o ID) después del operador >, pero se encontró 'THEN' [cerca de la posición 4 (línea 1, columna 23)].`
- *Omisión de `THEN`:*
  `Error sintáctico: se esperaba la palabra reservada THEN o un operador lógico (AND, OR) antes de 'bloquear' [...]`
- *Paréntesis sin cerrar:*
  `Error sintáctico: se esperaba el paréntesis de cierre ')' para`

`cerrar la agrupación [cerca de la posición 10 (línea 1, columna 43)].`

- *Falta de punto y coma:*
  `Error sintáctico: se esperaba ';' al final de la regla antes de finalizar la entrada [al final de la entrada (posición 8, línea 1)].`
- *Símbolo no reconocido (error léxico):*
  `Error léxico en línea 1, columna 25 (posición 5): símbolo no reconocido '@'.`

10. Casos de prueba

10.1. Extensiones creativas

Se integraron cuatro extensiones en la gramática, en FIRST/FOLLOW, en la tabla LL(1), en el parser y en las pruebas:

1. agrupación con paréntesis `( condicion )` con anidamiento arbitrario (producción 09);
2. operador unario `NOT` con precedencia superior a `AND` y `OR` (producción 08);
3. cláusula `ELSE` opcional (producción 16);
4. múltiples acciones separadas por coma y acciones con parámetro o con objeto (producciones 12–15).

10.2. Resultados

Se ejecutaron **30 casos** (12 válidos, 12 inválidos y 6 límite). En todos coincide el resultado esperado con el obtenido (30/30). La columna de observación muestra la nota del caso cuando es aceptado y el mensaje de error real emitido por el analizador cuando es rechazado. Los mismos resultados se obtienen con `python tests/test_suite.py`.

Cuadro 6: Casos de prueba.

ID	Entrada	Esperado	Obtenido	Observación
V01	`WHEN intentos_login >5 THEN bloquear ip_origen;`	Aceptada	Aceptada	Regla simple de fuerza bruta con operador '>' y acción verbo-objeto.
V02	`WHEN puerto_destino == 22 AND ip_verificada == FALSE THEN cerrar_puerto(22);`	Aceptada	Aceptada	Condición compuesta con AND, valor booleano FALSE y acción con parámetro numérico.
V03	`WHEN trafico_mb >= 1000 OR conexiones_activas >= 500 THEN limitar_ancho_banda(100), alertar soc_admin;`	Aceptada	Aceptada	Operador OR y múltiples acciones separadas por coma (extensión creativa).
V04	`WHEN NOT firewall_activo == TRUE THEN activar firewall_perimetral, registrar_incidente;`	Aceptada	Aceptada	Uso del operador unario NOT y combinación de acción con objeto y acción simple.

ID	Entrada	Esperado	Obtenido	Observación
V05	`WHEN (intentos_ssh >3 OR intentos_rdp >3) AND ip_interna == FALSE THEN aislar_host(ip_origen) ELSE registrar_acceso;`	Aceptada	Aceptada	Agrupación con paréntesis, precedencia OR/AND y cláusula condicional ELSE.
V06	`WHEN nivel_amenaza != 0 THEN bloquear_ip(ip_atacante), cerrar_puerto(445), notificar_admin(TRUE);`	Aceptada	Aceptada	Tres acciones encadenadas con parámetros de tipo ID, NUM y booleano TRUE.
V07	`WHEN paquetes_syn >10000 AND NOT (ip_whitelist == TRUE OR modo_mantenimiento == TRUE) THEN activar_mitigacion_ddos;`	Aceptada	Aceptada	Negación NOT aplicada sobre una agrupación entre paréntesis.
V08	`WHEN latencia_ms <= 50 AND perdida_paquetes <2 THEN mantener_ruta_principal ELSE conmutar_respaldo;`	Aceptada	Aceptada	Operadores relacionales '<=' y '<' con rama ELSE.
V09	`WHEN firma_malware == TRUE THEN poner_cuarentena archivo_sospechoso; WHEN riesgo_fuga >= 80 THEN bloquear_puerto(21);`	Aceptada	Aceptada	Programa compuesto por múltiples reglas consecutivas en secuencia.
V10	`WHEN payload_sql == TRUE OR (cabecera_xss == TRUE AND waf_activo == FALSE) THEN rechazar_peticion, bloquear_ip(cliente_http);`	Aceptada	Aceptada	Expresión anidada en segundo operando de OR con acciones múltiples.
V11	`WHEN certificado_vencido == TRUE THEN redirigir_trafico(443) ELSE permitir_conexion(80);`	Aceptada	Aceptada	Acciones parametrizadas tanto en la rama THEN como en la rama ELSE.
V12	`WHEN temperatura >28 AND humedad <40 THEN encender ventilador;`	Aceptada	Aceptada	Compatibilidad retroactiva con la sentencia de prueba original del notebook base.
I01	`intentos_login >5 THEN bloquear ip_origen;`	Rechazada	Rechazada	Error sintáctico: se esperaba la palabra reservada WHEN al inicio de la regla, pero se encontró 'intentos_login' [cerca de la posición 1 (línea 1, columna 1)].
I02	`WHEN intentos_login >THEN bloquear ip_origen;`	Rechazada	Rechazada	Error sintáctico: se esperaba un valor (NUM, TRUE, FALSE o ID) después del operador >, pero se encontró 'THEN' [cerca de la posición 4 (línea 1, columna 23)].
I03	`WHEN intentos_login >5 bloquear ip_origen;`	Rechazada	Rechazada	Error sintáctico: se esperaba la palabra reservada THEN o un operador lógico (AND, OR) antes de 'bloquear' [cerca de la posición 5 (línea 1, columna 25)].
I04	`WHEN intentos_login >5 THEN bloquear ip_origen`	Rechazada	Rechazada	Error sintáctico: se esperaba ';' al final de la regla antes de finalizar la entrada [al final de la entrada (posición 8, línea 1)].
I05	`WHEN intentos_login 5 THEN bloquear ip_origen;`	Rechazada	Rechazada	Error sintáctico: se esperaba un operador relacional (>, <, >=, <=, ==, !=) después del identificador 'intentos_login', pero se encontró '5' [cerca de la posición 3 (línea 1, columna 21)].
I06	`WHEN (intentos_login >5 AND puerto == 22 THEN bloquear ip_origen;`	Rechazada	Rechazada	Error sintáctico: se esperaba el paréntesis de cierre ')' para cerrar la agrupación [cerca de la posición 10 (línea 1, columna 43)].
I07	`WHEN intentos_login >5 AND THEN bloquear ip_origen;`	Rechazada	Rechazada	Error sintáctico: se esperaba una condición (identificador, NOT o '(') después de 'AND', pero se encontró 'THEN' [cerca de la posición 6 (línea 1, columna 29)].

ID	Entrada	Esperado	Obtenido	Observación
I08	`WHEN intentos_login >5 THEN ;`	Rechazada	Rechazada	Error sintáctico: se esperaba un identificador de acción después de 'THEN', pero se encontró ';' [cerca de la posición 6 (línea 1, columna 30)].
I09	`WHEN intentos_login >5 THEN bloquear_puerto();`	Rechazada	Rechazada	Error sintáctico: se esperaba un parámetro dentro de 'bloquear_puerto(...)' antes de ')' [cerca de la posición 8 (línea 1, columna 46)].
I10	`WHEN intentos_login >5 THEN bloquear ip_origen, ;`	Rechazada	Rechazada	Error sintáctico: se esperaba un identificador de acción después de ',', pero se encontró ';' [cerca de la posición 9 (línea 1, columna 50)].
I11	`WHEN intentos_login >5 THEN bloquear ip_origen ELSE ;`	Rechazada	Rechazada	Error sintáctico: se esperaba un identificador de acción después de 'ELSE', pero se encontró ';' [cerca de la posición 9 (línea 1, columna 54)].
I12	`WHEN intentos_login >5 @ puerto == 22 THEN bloquear ip_origen;`	Rechazada	Rechazada	Error léxico en línea 1, columna 25 (posición 5): símbolo no reconocido '@'.
L01	(vacía)	Rechazada	Rechazada	Error sintáctico: la entrada está vacía; se esperaba al menos una regla iniciada con la palabra reservada WHEN [al final de la entrada (posición 1, línea 1)].
L02	`((WHEN ((puerto == 80)) THEN auditar));`	Rechazada	Rechazada	Error sintáctico: se esperaba la palabra reservada WHEN al inicio de la regla, pero se encontró '(' [cerca de la posición 1 (línea 1, columna 1)].
L03	`WHEN (((NOT NOT alerta_critica == TRUE))) THEN aislar_nodo;`	Aceptada	Aceptada	Caso límite válido: triple anidamiento de paréntesis y doble negación recursiva NOT NOT.
L04	`when trafico_mb >0 then registrar_log;`	Aceptada	Aceptada	Caso límite válido: palabras reservadas en minúsculas (insensibilidad a mayúsculas/minúsculas).
L05	`# Regla comentada / WHEN puerto == 443 THEN permitir_tls; // Comentario final`	Aceptada	Aceptada	Caso límite válido: programa multilínea con comentarios '#' y '//' intercalados.
L06	`WHEN a == b AND c != d AND e >= f OR g <= h THEN acc1, acc2(1), acc3 obj3, acc4(TRUE) ELSE acc5;`	Aceptada	Aceptada	Caso límite válido: combinación máxima de todos los operadores, tipos de acciones y rama ELSE.

11. Conclusiones

1. **Coherencia entre teoría e implementación.** El diseño de CyberGuard DSL mostró cómo las propiedades de una gramática libre de contexto determinan la construcción de un analizador determinista. La estratificación por precedencia (`OR`, `AND`, `NOT`, paréntesis), junto con la eliminación de recursión izquierda y la factorización, permitió obtener una tabla LL(1) sin conflictos.

2. **Modularidad y diagnóstico de errores.** Separar el análisis léxico, la definición formal de la gramática, el parser y la visualización facilitó generar árboles y entregar mensajes de error localizados, que indican lo esperado, lo encontrado y la posición.

3. **Dos parsers equivalentes.** Implementar el descenso recursivo y el parser dirigido por tabla sobre la misma gramática permitió contrastar ambos enfoques y comprobar que la tabla de la sección 6 coincide con el comportamiento del código.

4. **Limitaciones.** El lenguaje solo admite enteros no negativos y no incluye cadenas ni

expresiones aritméticas. Como mejoras futuras se proponen operadores aritméticos, cadenas (direcciones IP literales) y recuperación de errores para informar varios errores en una misma ejecución.

12. Anexos

12.1. Repositorio

Código fuente, README, archivos auxiliares y salidas:
`https://github.com/eduardoscrs/Analisis-Sintactico-INFO1148`

12.2. Estructura del repositorio

```
Analisis-Sintactico-INFO1148/
|-- README.md
|-- main.py
|-- src/
|   |-- lexer.py
|   |-- grammar_ll1.py
|   |-- parser.py
|   `-- ast_visualizer.py
|-- tests/test_suite.py
|-- notebooks/anlizador_lexico1.ipynb
|-- docs/        (enunciado, rubrica e informe)
`-- outputs/     (arboles .svg y .dot)
```

12.3. Demostración funcional

Las siguientes capturas muestran la salida real de la ejecución del programa (los comandos aparecen en la primera línea de cada captura).

```
$ python main.py -e "WHEN (intentos_ssh > 3 OR puerto_destino == 22) AND NOT ... ELSE registrar_acceso;"

================================================================================================
4. DEMOSTRACIÓN DE ANÁLISIS LÉXICO, SINTÁCTICO Y GENERACIÓN DE ÁRBOLES
================================================================================================
Entrada analizada:
  WHEN (intentos_ssh > 3 OR puerto_destino == 22) AND NOT ip_confiable == TRUE THEN bloquear_ip(ip_origen), alertar
soc_admin ELSE registrar_acceso;

Estado: Entrada sintácticamente válida (Aceptada).

Tokens reconocidos [(TIPO, lexema, línea, columna)]:
  (WHEN       , 'WHEN', lín=1, col=1)
  (PARENT_IZQ , '(', lín=1, col=6)
  (ID         , 'intentos_ssh', lín=1, col=7)
  (OP_REL     , '>', lín=1, col=20)
  (NUM        , '3', lín=1, col=22)
  (OR         , 'OR', lín=1, col=24)
  (ID         , 'puerto_destino', lín=1, col=27)
  (OP_REL     , '==', lín=1, col=42)
  (NUM        , '22', lín=1, col=45)
  (PARENT_DER , ')', lín=1, col=47)
  (AND        , 'AND', lín=1, col=49)
  (NOT        , 'NOT', lín=1, col=53)
  (ID         , 'ip_confiable', lín=1, col=57)
  (OP_REL     , '==', lín=1, col=70)
  (TRUE       , 'TRUE', lín=1, col=73)
  (THEN       , 'THEN', lín=1, col=78)
  (ID         , 'bloquear_ip', lín=1, col=83)
  (PARENT_IZQ , '(', lín=1, col=94)
  (ID         , 'ip_origen', lín=1, col=95)
  (PARENT_DER , ')', lín=1, col=104)
  (COMA       , ',', lín=1, col=105)
  (ID         , 'alertar', lín=1, col=107)
  (ID         , 'soc_admin', lín=1, col=115)
  (ELSE       , 'ELSE', lín=1, col=125)
  (ID         , 'registrar_acceso', lín=1, col=130)
  (PUNTO_COMA , ';', lín=1, col=146)

--- ÁRBOL DE SINTAXIS ABSTRACTA (AST) ---
Programa
└── Regla
    ├── Condicion
    │   └── OpLogico: AND
    │       ├── OpLogico: OR
    │       │   ├── Comparacion: >
```

Figura 1: Análisis léxico y sintáctico de una regla válida con `python main.py -e`.

```
$ python main.py -e "WHEN (intentos_ssh > 3 OR ...) ... ELSE registrar_acceso;"   # salida: AST
--- ÁRBOL DE SINTAXIS ABSTRACTA (AST) ---
Programa
└── Regla
    ├── Condicion
    │   └── OpLogico: AND
    │       ├── OpLogico: OR
    │       │   ├── Comparacion: >
    │       │   │   ├── Identificador: intentos_ssh
    │       │   │   └── Numero: 3
    │       │   └── Comparacion: ==
    │       │       ├── Identificador: puerto_destino
    │       │       └── Numero: 22
    │       └── OpUnario: NOT
    │           └── Comparacion: ==
    │               ├── Identificador: ip_confiable
    │               └── Booleano: TRUE
    ├── AccionesTHEN
    │   ├── Accion: bloquear_ip
    │   │   └── Parametro
    │   │       └── Identificador: ip_origen
    │   └── Accion: alertar
    │       └── Objetivo: soc_admin
    └── AccionesELSE
        └── Accion: registrar_acceso
```

Figura 2: Árbol de sintaxis abstracta (AST) generado para la regla válida.

```
$ python main.py -e "WHEN intentos_login > THEN bloquear ip_origen;"

================================================================================================
4. DEMOSTRACIÓN DE ANÁLISIS LÉXICO, SINTÁCTICO Y GENERACIÓN DE ÁRBOLES
================================================================================================
Entrada analizada:
  WHEN intentos_login > THEN bloquear ip_origen;

Estado: Error sintáctico: se esperaba un valor (NUM, TRUE, FALSE o ID) después del operador >, pero se
encontró 'THEN' [cerca de la posición 4 (línea 1, columna 23)].
```

Figura 3: Entrada inválida: mensaje de error sintáctico localizado.

```
$ python src/grammar_ll1.py
=== CONJUNTOS FIRST Y FOLLOW ===
<programa>             | FIRST = ['WHEN'] | FOLLOW = ['$']
<resto_programa>       | FIRST = ['WHEN', 'ε'] | FOLLOW = ['$']
<regla>                | FIRST = ['WHEN'] | FOLLOW = ['$', 'WHEN']
<condicion>            | FIRST = ['ID', 'NOT', 'PARENT_IZQ'] | FOLLOW = ['PARENT_DER', 'THEN']
<resto_or>             | FIRST = ['OR', 'ε'] | FOLLOW = ['PARENT_DER', 'THEN']
<termino_and>          | FIRST = ['ID', 'NOT', 'PARENT_IZQ'] | FOLLOW = ['OR', 'PARENT_DER', 'THEN']
<resto_and>            | FIRST = ['AND', 'ε'] | FOLLOW = ['OR', 'PARENT_DER', 'THEN']
<factor_not>           | FIRST = ['ID', 'NOT', 'PARENT_IZQ'] | FOLLOW = ['AND', 'OR', 'PARENT_DER', 'THEN']
<primario>             | FIRST = ['ID', 'PARENT_IZQ'] | FOLLOW = ['AND', 'OR', 'PARENT_DER', 'THEN']
<comparacion>          | FIRST = ['ID'] | FOLLOW = ['AND', 'OR', 'PARENT_DER', 'THEN']
<valor>                | FIRST = ['FALSE', 'ID', 'NUM', 'TRUE'] | FOLLOW = ['AND', 'OR', 'PARENT_DER', 'THEN']
<lista_acciones>       | FIRST = ['ID'] | FOLLOW = ['ELSE', 'PUNTO_COMA']
<resto_acciones>       | FIRST = ['COMA', 'ε'] | FOLLOW = ['ELSE', 'PUNTO_COMA']
<accion>               | FIRST = ['ID'] | FOLLOW = ['COMA', 'ELSE', 'PUNTO_COMA']
<complemento_accion>   | FIRST = ['ID', 'PARENT_IZQ', 'ε'] | FOLLOW = ['COMA', 'ELSE', 'PUNTO_COMA']
<clausula_else>        | FIRST = ['ELSE', 'ε'] | FOLLOW = ['PUNTO_COMA']

¿Es LL(1) sin conflictos?: True
Total de entradas en la tabla LL(1): 39
```

Figura 4: Conjuntos FIRST y FOLLOW calculados por `src/grammar_ll1.py` y verificación de que la gramática es LL(1).

```
$ python tests/test_suite.py
================================================================================================================================================
ID   | Categoría   | Esperado   | Obtenido   | Estado | Observación / Mensaje del Analizador
================================================================================================================================================
V01  | Válida      | Aceptada   | Aceptada   | OK     | Regla simple de fuerza bruta con operador '>' y acción verbo-objeto.
V02  | Válida      | Aceptada   | Aceptada   | OK     | Condición compuesta con AND, valor booleano FALSE y acción con parámetro numérico.
V03  | Válida      | Aceptada   | Aceptada   | OK     | Operador OR y múltiples acciones separadas por coma (extensión creativa).
V04  | Válida      | Aceptada   | Aceptada   | OK     | Uso del operador unario NOT y combinación de acción con objeto y acción simple.
V05  | Válida      | Aceptada   | Aceptada   | OK     | Agrupación con paréntesis, precedencia OR/AND y cláusula condicional ELSE.
V06  | Válida      | Aceptada   | Aceptada   | OK     | Tres acciones encadenadas con parámetros de tipo ID, NUM y booleano TRUE.
V07  | Válida      | Aceptada   | Aceptada   | OK     | Negación NOT aplicada sobre una agrupación entre paréntesis.
V08  | Válida      | Aceptada   | Aceptada   | OK     | Operadores relacionales '<=' y '<' con rama ELSE.
V09  | Válida      | Aceptada   | Aceptada   | OK     | Programa compuesto por múltiples reglas consecutivas en secuencia.
V10  | Válida      | Aceptada   | Aceptada   | OK     | Expresión anidada en segundo operando de OR con acciones múltiples.
V11  | Válida      | Aceptada   | Aceptada   | OK     | Acciones parametrizadas tanto en la rama THEN como en la rama ELSE.
V12  | Válida      | Aceptada   | Aceptada   | OK     | Compatibilidad retroactiva con la sentencia de prueba original del notebook base.
I01  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba la palabra reservada WHEN al inicio de la regla, pero se encontró 'intentos_login' [cerca de la posición 1 (línea 1, columna 1)].
I02  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba un valor (NUM, TRUE, FALSE o ID) después del operador >, pero se encontró 'THEN' [cerca de la posición 4 (línea 1, columna 23)].
I03  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba la palabra reservada THEN o un operador lógico (AND, OR) antes de 'bloquear' [cerca de la posición 5 (línea 1, columna 25)].
I04  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba ';' al final de la regla antes de finalizar la entrada [al final de la entrada (posición 8, línea 1)].
I05  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba un operador relacional (>, <, >=, <=, ==, !=) después del identificador 'intentos_login', pero se encontró '5' [cerca de la posición 3 (línea 1, columna 21)].
I06  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba el paréntesis de cierre ')' para cerrar la agrupación [cerca de la posición 10 (línea 1, columna 43)].
I07  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba una condición (identificador, NOT o '(') después de 'AND', pero se encontró 'THEN' [cerca de la posición 6 (línea 1, columna 29)].
I08  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba un identificador de acción después de 'THEN', pero se encontró ';' [cerca de la posición 6 (línea 1, columna 30)].
I09  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba un parámetro dentro de 'bloquear_puerto(...)' antes de ')' [cerca de la posición 8 (línea 1, columna 46)].
I10  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba un identificador de acción después de ',', pero se encontró ';' [cerca de la posición 9 (línea 1, columna 50)].
I11  | Inválida    | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba un identificador de acción después de 'ELSE', pero se encontró ';' [cerca de la posición 9 (línea 1, columna 54)].
I12  | Inválida    | Rechazada  | Rechazada  | OK     | Error léxico en línea 1, columna 25 (posición 5): símbolo no reconocido '@'.
L01  | Caso Límite | Rechazada  | Rechazada  | OK     | Error sintáctico: la entrada está vacía; se esperaba al menos una regla iniciada con la palabra reservada WHEN [al final de la entrada (posición 1, línea 1)].
L02  | Caso Límite | Rechazada  | Rechazada  | OK     | Error sintáctico: se esperaba la palabra reservada WHEN al inicio de la regla, pero se encontró '(' [cerca de la posición 1 (línea 1, columna 1)].
L03  | Caso Límite | Aceptada   | Aceptada   | OK     | Caso límite válido: triple anidamiento de paréntesis y doble negación recursiva NOT NOT.
L04  | Caso Límite | Aceptada   | Aceptada   | OK     | Caso límite válido: palabras reservadas en minúsculas (insensibilidad a mayúsculas/minúsculas).
L05  | Caso Límite | Aceptada   | Aceptada   | OK     | Caso límite válido: programa multilínea con comentarios '#' y '//' intercalados.
L06  | Caso Límite | Aceptada   | Aceptada   | OK     | Caso límite válido: combinación máxima de todos los operadores, tipos de acciones y rama ELSE.
================================================================================================================================================
Resumen de pruebas: 30/30 casos superados exitosamente (100%).
```

Figura 5: Ejecución de la batería de pruebas: 30/30 casos superados.